# Benchmarking Post-Quantum Key Encapsulation in S3-Compatible Object Storage under Client Concurrency

Team 12 (author names to be added)
Department and institution to be added

***Abstract*—Large-scale quantum computers would break the RSA and elliptic-curve key establishment that protects most encrypted cloud data, and ciphertext stored today can be decrypted later. NIST standardized ML-KEM (FIPS 203) as the replacement key-encapsulation mechanism, but its cost has mostly been measured in isolation or inside TLS handshakes, not in the per-object encryption path of cloud object stores under concurrent load. We design and implement an instrumented client-side envelope-encryption layer for an S3-compatible store (MinIO) in which every object is protected by ML-KEM-768, HKDF-SHA256 and chunked AES-256-GCM, and compare it on one data path with no encryption, symmetric-only encryption, RSA-OAEP-4096, X25519 and a hybrid X25519+ML-KEM-768 mode, from 1 to 500 concurrent workers and from 1 KiB to 500 MiB objects, on two separate cloud instances (1,970 runs, 101.3 million measured requests, 0 failures). ML-KEM-768 costs 0.045 ms per PUT and 0.047 ms per GET, at most 2.25 % of request time even for the smallest objects. In 38 of 44 paired comparisons its throughput is statistically equivalent to X25519 within $\pm 5$ %, and none is confirmed to differ by more (6 are inconclusive). Every mode reaches the same network, storage or client-CPU ceiling, except RSA-4096, whose 1.486 ms decryption saturates the client and cuts small-object GET throughput to 0.38–0.59 of ML-KEM's. Bounded session amortization of ML-KEM saves 30–66 µs of client CPU per request at one worker and up to 100 µs under contention, which raises throughput only where the client is the bottleneck. Open-loop load shows a sharp knee between $\rho = 0.58$ and $0.83$ that an M/M/c model does not predict.**

***Index Terms*—post-quantum cryptography, ML-KEM, object storage, S3, envelope encryption, performance evaluation, benchmarking**

## I. Introduction

Cloud object stores keep data for years, and much of that data is encrypted under keys established with RSA or elliptic-curve Diffie–Hellman. Shor's algorithm would allow a sufficiently large quantum computer to recover those keys, so an adversary who records ciphertext today can decrypt it once such a machine exists, a threat usually called "harvest now, decrypt later". For data at rest the exposure lasts as long as the data is retained, which makes storage an early candidate for migration. In August 2024 NIST published FIPS 203, standardizing the module-lattice key-encapsulation mechanism ML-KEM and recommending ML-KEM-768 as its default parameter set [1].

Moving object storage to ML-KEM is not a drop-in change. With client-side envelope encryption, every object needs its own key establishment, so the cost of the KEM is paid on every PUT and every GET rather than once per connection. Migration raises three practical questions: (a) the CPU cost of encapsulation and decapsulation per request; (b) the larger key-establishment ciphertext, which must be stored with every object; and (c) the interaction of both with concurrency, where many clients share CPUs, the network and the storage servers. Earlier performance studies of post-quantum cryptography concentrate on algorithm micro-benchmarks and on TLS connection establishment [2], [3], where the cost is amortized over a session; performance studies of the S3 interface, in turn, do not include cryptography [4]. To our knowledge, the per-object path of an object store under concurrent load has not been measured.

In this paper, we measure what per-object ML-KEM-768 costs an S3-compatible object store and when that cost becomes visible. We ask three research questions. **RQ1:** What does per-object ML-KEM-768 add to latency, throughput, CPU and memory, compared with RSA-4096, X25519 and symmetric-only baselines on the same data path? **RQ2:** How does this change with object size and with 1–500 concurrent workers? **RQ3:** Can a queueing model predict where response time degrades, and can bounded session amortization of ML-KEM move that point?

The **key contributions** of this work are as follows:

1) We design a client-side envelope-encryption layer for S3-compatible stores with six interchangeable key-establishment modes on one data path, chunked authenticated encryption bound to the object's identity, and a commit protocol that never exposes partial uploads (§IV).
2) We implement it, together with a measurement harness that decomposes every request into key establishment, key derivation, encryption, metadata and storage time, samples client and server resources, and refuses inconsistent data; and we automate a two-host cloud testbed (§V).
3) We report a measurement of 1,970 runs with five randomized repetitions per configuration, paired within-batch comparisons and explicit equivalence tests, covering handshake overhead, throughput, CPU and memory (§VII).
4) We evaluate bounded session amortization of ML-KEM, including a variant whose bound follows client utilization, and test an M/M/c model against open-loop

measurements.

Our **key findings** are: (i) ML-KEM-768 costs 0.045 ms per PUT and 0.047 ms per GET, less than X25519 on PUT and 32× less than RSA-4096 decryption on GET; (ii) its throughput is equivalent to X25519 within ±5 % in 38 of 44 comparisons, because every workload is limited by the network, the storage server or the client CPU, not by the KEM; (iii) RSA-4096 is the one mode that changes the bottleneck, limiting small-object reads to 0.38–0.59 of ML-KEM's throughput; (iv) the measurable price of ML-KEM is 1,412 more bytes of metadata per object than X25519; and (v) response time collapses well before nominal saturation, a knee that an M/M/c model with load-independent service time does not predict.

**Scope.** We do not evaluate the security of ML-KEM itself, side channels of the implementation, or transport security, which we hold constant. Bounded key reuse is established practice (§III); our claim for session amortization is its application to ML-KEM roots, its quantified trade-off, and the queue-aware variant as a candidate policy.

**Organization.** Section II gives background and Section III discusses related work. Section IV presents the threat model and design, Section V the implementation and Section VI the experimental methodology. Section VII reports the results, Section VIII discusses their implications, Section IX the threats to validity, and Section X concludes.

## II. Background

### A. Key-Encapsulation Mechanisms and ML-KEM

A key-encapsulation mechanism (KEM) consists of three algorithms: key generation; encapsulation, which takes the recipient's public (encapsulation) key and outputs a ciphertext and a shared secret; and decapsulation, which recovers the shared secret from the ciphertext with the private (decapsulation) key. ML-KEM, derived from CRYSTALS-Kyber, bases its security on the hardness of the module learning-with-errors problem. ML-KEM-768 targets NIST security category 3 with a 1,184-byte encapsulation key, a 2,400-byte decapsulation key, a 1,088-byte ciphertext and a 32-byte shared secret [1]. Decapsulation uses *implicit rejection*: a modified ciphertext or a wrong key yields a pseudorandom secret instead of an error, so tampering is detected by the authenticated encryption that follows.

### B. Envelope Encryption

In envelope encryption a per-object data key encrypts the object, and key establishment protects that key for the recipient. We derive data keys with HKDF-SHA256 [5] and encrypt with AES-256-GCM, whose security requires that a nonce is never reused under the same key [6]. The classical alternatives for key establishment are key transport with RSA-OAEP [7] and ephemeral-static Diffie–Hellman such as X25519 [8].

### C. S3-Compatible Object Storage

Object stores expose a flat namespace of buckets and keys through HTTP operations, most importantly PUT and GET of whole objects; large objects are uploaded in parts. The S3 interface has become the de facto standard, and many systems implement it [4]. We use MinIO [9]. Its community edition was archived in 2026 and no longer publishes images, so we compile the final community release (RELEASE.2025-10-15T17-29-55Z) from source; because all cryptography is client-side, any S3-compatible store can be substituted.

### D. Queueing Model

An M/M/c queue has Poisson arrivals at rate $\lambda$, $c$ identical servers and exponentially distributed service times with mean $S$ [10]. With offered load $a = \lambda S$ and utilization $\rho = a/c$, the probability that an arrival must wait is given by the Erlang C formula $C(c, a)$, and the mean response time is $W = S + C(c, a)\, S/(c - a)$ for $\rho < 1$. We use this model in RQ3 to ask whether response time under open-loop load can be predicted from the capacity and the light-load service time alone.

## III. Related Work

**Algorithm-level cost.** Library benchmarks such as those of liboqs [11] and comparative studies of ML-KEM against RSA and elliptic curves [12] quantify key generation, encapsulation and decapsulation and the larger key and ciphertext sizes. They do not include storage, concurrency or end-to-end request latency.

**Post-quantum TLS.** Paquin et al. [2] measure TLS 1.3 connection establishment with hybrid post-quantum key exchange and signatures under emulated network latency and packet loss, and Sikeridis et al. [3] study post-quantum authentication in TLS 1.3, including its effect on server throughput. Hybrid combiners such as X-Wing [13] target the same setting. These works concern data in transit, where key establishment happens once per connection; our work concerns data at rest, where it happens once per object, and holds transport security constant.

**Object-storage performance.** Gadban and Kunkel [4] analyze the performance of the S3 interface for HPC workloads and attribute losses to layers of the client stack. Our harness follows the same principle of separating storage time from client-side cost, applied to the cryptographic stages.

**Post-quantum cloud storage.** Prikutse et al. [14] combine AES-256-GCM, X25519+ML-KEM-768 and ML-DSA-65 for multi-file cloud storage and amortize signatures by batch commitment. According to its abstract, that evaluation does not cover an S3 store, concurrent clients or queueing behaviour.

**Bounded key reuse.** Amazon S3 Bucket Keys derive per-object data keys from a time-limited bucket-level key and reduce key-service requests by up to 99% [15]; the AWS Encryption SDK caches data keys with age and message-count limits [16]. Our session mode applies the same principle to ML-KEM-established roots in client-side encryption. In vendor documentation and arXiv we found no prior load-adaptive bound; a search of the academic databases is still to be completed.

TABLE I
KEY-ESTABLISHMENT MODES COMPARED. SIZES IN BYTES; THE PER-OBJECT VALUE IS WHAT EVERY OBJECT'S MANIFEST CARRIES.

| Mode | Key establishment | Hard problem | Public key | Per object | PQ |
|---|---|---|---|---|---|
| B0 | none (plaintext) | – | – | – | – |
| B1 | pre-shared key | – | – | 0 | yes |
| R1 | RSA-OAEP-4096 | factoring | 512 | 512 | no |
| B2 | X25519 | ECDLP | 32 | 32 | no |
| P1 | ML-KEM-768 | MLWE | 1,184 | 1,088 | yes |
| P2 | X25519+ML-KEM-768 | both | 1,216 | 1,120 | yes |
| N1 | ML-KEM-768 session | MLWE | 1,184 | 1,088 | yes |

## IV. THREAT MODEL AND DESIGN

### A. Threat Model

The network and the object store are honest-but-curious for confidentiality and potentially malicious for integrity: an adversary may read, replace, delete, reorder or replay stored ciphertexts and manifests. The adversary does not hold the recipient's decapsulation key and may in the future operate a quantum computer. Availability, traffic analysis and side channels in the client are out of scope.

### B. Key-Establishment Modes

Table I lists the seven modes. All encrypting modes share the HKDF and chunked AES-GCM path, so differences between them isolate the cost of key establishment. B0 stores plaintext through the same framing and measures the storage and harness cost; B1 derives keys from a pre-shared 32-byte key and is the symmetric-only floor; R1 transports a random 32-byte secret with RSA-OAEP-4096; B2 uses ephemeral-static X25519; P1 uses ML-KEM-768; P2 is a hybrid that stores the ML-KEM ciphertext followed by the X25519 ephemeral key and feeds both 32-byte secrets to HKDF; and N1 amortizes one ML-KEM encapsulation over a bounded session (§IV-D). A reader must name the mode it expects, so a planted plaintext object cannot downgrade P1.

### C. Per-Object Envelope and Commit Protocol

Algorithm 1 shows PUT and GET for mode P1. An object is identified by $I = (tenant, bucket, key, version)$, where versions are time-ordered UUIDv7 values. The context $ctx$ is canonical JSON of $I$, the mode, the algorithm names, chunk size and count, plaintext size, recipient key identifier and the SHA-256 hash of the key-establishment ciphertext $C$; it is bound into the data key and into every chunk. Objects are split into 4 MiB chunks, and chunk $j$ is encrypted with nonce $p\|\mathrm{be}_{64}(j)$ under a random 32-bit prefix $p$ and associated data $ctx\|j\|\ell_j\|final_j$, so deletion, reordering, truncation, extension and cross-object splicing all fail authentication. Each version is stored under `key.pqcs/version/` as a ciphertext stream `data.enc` and a manifest `manifest.json` holding $ctx$, $C$, the salt and the prefix. The manifest is written last and serves as the commit record: readers only consider versions whose manifest exists, so an interrupted upload is never visible.

**Algorithm 1** Per-object PUT and GET (mode P1)

**procedure** PUT($key, M, ek$)
  $v \leftarrow$ UUIDv7(); $I \leftarrow (tenant, bucket, key, v)$
  $(C, S) \leftarrow$ ML-KEM.Encaps($ek$)
  $salt \xleftarrow{\$} \{0,1\}^{256}$; $p \xleftarrow{\$} \{0,1\}^{32}$
  $ctx \leftarrow$ CANON($I$, mode, $n$, $|M|$, id($ek$), H($C$), ...)
  $K \leftarrow$ HKDF($S$, $salt$, "pqc-s3-object-v2"$\|0\|ctx$)
  **for** $j \leftarrow 0$ **to** $n-1$ **do** ▷ streamed in 8 MiB parts
    $c_j \leftarrow$ AES-GCM$_K(p\|\mathrm{be}_{64}(j), M_j, ctx\|j\|\ell_j\|final_j)$
  **end for**
  write $c_0\|\ldots\|c_{n-1}$ to `key.pqcs/v/data.enc`
  commit: write $(ctx, C, salt, p)$ to `key.pqcs/v/manifest.json`
**end procedure**
**procedure** GET($key, dk$, mode)
  $v \leftarrow$ newest version with a manifest; read and parse it
  **reject** unless identity, mode, key id and H($C$) all match
  $S \leftarrow$ ML-KEM.Decaps($dk, C$); $K \leftarrow$ HKDF($S$, $salt$, ...)
  **for** each chunk $c_j$ of `data.enc` **do**
    $M_j \leftarrow$ AES-GCM$_K^{-1}$(...); **reject** on failure
  **end for**
  **reject** unless exactly $n$ chunks were read and the last is final
**end procedure**

### D. Bounded Sessions (Mode N1)

In N1 one encapsulation yields a root $S_r$, and object $i$ of the session uses $K_i = \mathrm{HKDF}(S_r, salt_i, \text{"pqc-s3-session-v1"}\|0\|ctx_i)$, with the session identifier, the object's sequence number and the bound authenticated in $ctx_i$. Every object still stores the session's ML-KEM ciphertext, so it can be decrypted alone. A root protects at most $B$ objects and lives at most $T_{\max}$ seconds. In the queue-aware variant, $B_t = \mathrm{clip}(B_{\min}, B_{\max}, \mathrm{round}(B_{\min} + (B_{\max} - B_{\min})\hat{\rho}))$, where $\hat{\rho}$ is an exponentially weighted moving average ($\alpha = 0.2$) of the fraction of a client process's worker threads with a request in flight: under light load a root is used for fewer objects. $B_t$ is fixed per session, so $B_{\max}$ and $T_{\max}$ are hard caps. A leaked root exposes at most $B_{\max}$ objects; compromise of the recipient's decapsulation key exposes all objects in both P1 and N1. Receivers cache decapsulated roots under their own caps and reject objects whose authenticated sequence number exceeds them.

## V. IMPLEMENTATION

Figure 1 shows the system and Table II its components. The client, harness and analysis are written in Python 3.12.3; in total the project comprises 5,970 non-blank lines of code, including 220 automated tests.

### A. Cryptographic Core

ML-KEM-768 comes from liboqs 0.16.0 through liboqs-python 0.16.0.1 [11], [17]. We build liboqs from a pinned release with only the ML-KEM parameter sets enabled and load it from an explicit installation path, so the benchmark cannot silently pick up a different library; a start-up check verifies the library version and that ML-KEM-768 is enabled, and every call checks key and ciphertext sizes. RSA-OAEP-4096 (SHA-256), X25519, HKDF and AES-256-GCM come from pyca/cryptography 50.0.1 [18]. A registry maps each mode to its key-establishment and recovery functions, and

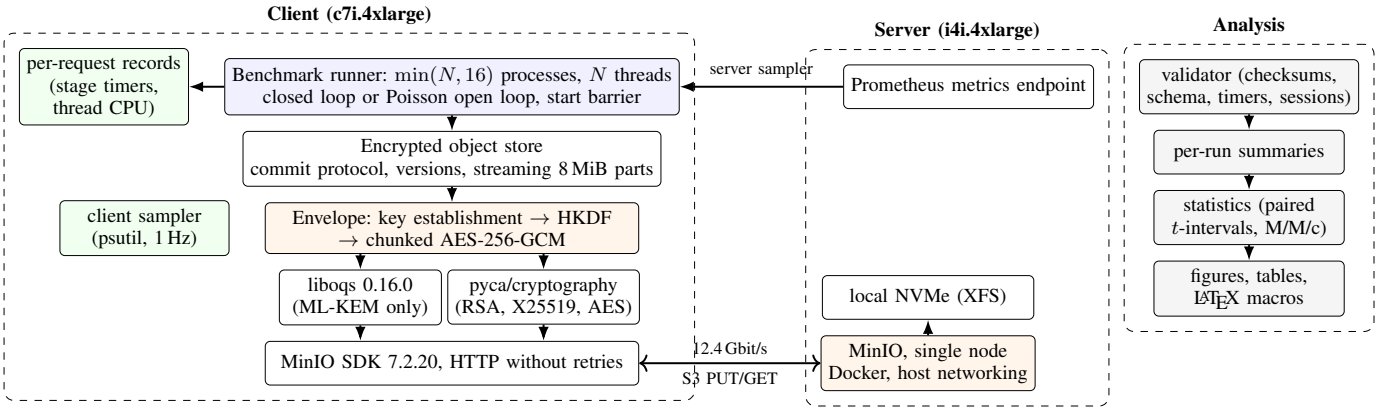



Fig. 1. System overview. The client stack (left) issues S3 requests to MinIO (centre); every request produces a record, and client and server resources are sampled once per second. Raw data passes through a validator before any analysis (right), which generates every number in this paper.

TABLE II
COMPONENTS OF THE IMPLEMENTATION (NON-BLANK LINES, COUNTED BY THE ANALYSIS PIPELINE)

| Component | Responsibility | Lines |
|---|---|---|
| Cryptographic core | ML-KEM wrapper, modes, HKDF, chunked AES-GCM, manifest, sessions | 826 |
| Storage layer | streaming PUT/GET, commit protocol, versions, garbage collection | 298 |
| Benchmark harness | runner, payloads, samplers, records, validator, micro-benchmarks | 1,094 |
| Command-line tool | keys, settings and the `pqcstore` commands | 412 |
| Testbed automation | AWS provisioning, VM roles, MinIO image | 537 |
| Analysis pipeline | per-run summaries, statistics, paper assets | 1,620 |
| Tests | unit, interoperability, tamper, secret-hygiene and integration tests | 1,183 |

the envelope module turns the result into a streaming chunk encryptor or decryptor, so the storage code never holds a whole object. Before any timed operation every worker process runs a warm-up that loads the libraries, performs a key generation, encapsulation, decapsulation, derivation and encryption round trip, and one full round trip in the mode under test, so that one-off costs never enter a timer. An interoperability test checks ML-KEM in both directions against an independent implementation (OpenSSL through pyca), and the manifest parser rejects unknown or missing fields, wrong types, inconsistent sizes and a key-establishment ciphertext that does not match its hash.

### B. Storage Layer

The storage layer implements the commit protocol of Algorithm 1 on the MinIO Python SDK 7.2.20. Ciphertext records are passed to the SDK through a file-like adapter over the encryptor's output, so an object is uploaded in 8 MiB parts while it is being encrypted; on GET the stream is decrypted chunk by chunk into a sink. For benchmarking, the HTTP client uses a connection pool sized to the worker threads and performs no automatic retries, so failures are counted rather than hidden inside latency. Auxiliary functions list committed versions, verify a commit by reading it back, delete versions and collect uncommitted data left behind by interrupted uploads.

### C. Measurement Harness

A *run* executes one configuration (mode, operation, object size, concurrency and loop type) with freshly started worker processes; it is the unit of replication. Concurrency $N$ is realized as exactly $N$ worker threads spread over $\min(N, 16)$ processes, one per vCPU. Each process warms up, reports readiness, and all of them start at a common wall-clock instant, so that no worker measures while others are still starting. Each run has a 5 s warm-up phase and a 20 s measured phase; Python's garbage collector is run before and disabled during the timed window, so its pauses cannot land inside a stage timer. In closed-loop runs each thread issues requests back to back. In open-loop runs each process draws Poisson arrivals at its share of the target rate, a request waits in a queue for a free thread, and the wait is recorded separately; a request that cannot start before the end of the run is recorded as a failure, never dropped. GET runs read a set of pre-written fixture objects that are created before and deleted after the run, and every object written by a PUT run is deleted afterwards, outside the timed window.

Every request, including warm-up requests and failures, produces one record with `perf_counter` timers for the five stages of Algorithm 1—key establishment, key derivation, AES-GCM, manifest handling and storage—its total latency, the thread CPU time it consumed, bytes on the wire and metadata size, plus identifiers that tie it to its run, repetition, code commit and configuration. A client sampler records the CPU time and resident memory of the worker processes and the host's CPU and network counters once per second; a server sampler reads the same information from MinIO's Prometheus metrics endpoint.

### D. Validation

A validator checks every data set before analysis and refuses it if any check fails: checksums of all files; a complete schema with correct types for every record; that no request's stage times exceed its total; that per run the 99th percentile of unaccounted time stays below 2 % of the request time and below 1 % in aggregate; and that no N1 session exceeds its bound. The 220 automated tests cover the cryptographic core, the commit protocol, tampering (bit flips; deleted, reordered, duplicated, truncated or extended chunks; swapped, replayed or relabelled manifests; mode downgrade), that manifests contain no secret material and private-key files are protected, session bounds under concurrent stress, and end-to-end runs against a live MinIO.

### E. Testbed Automation

A single script provisions and operates the cloud testbed: it creates both instances in one cluster placement group with a firewall that admits SSH only from the operator's address, copies the committed code, builds MinIO from source into a Docker image, formats the server's local NVMe disk at boot, runs network (iperf3, ping) and disk (fio) baselines, starts experiments in a detached session, fetches results and stops or deletes all resources. Before launch, a planner estimates the run time and cost of each experiment configuration. To bound cost, both instances stop themselves when idle, and the client stops itself shortly after an experiment completes.

### F. Analysis Pipeline

Analysis is a three-stage pipeline. A summarizer streams the compressed request records of each experiment once and writes one row per run (latency percentiles, stage means, throughput, CPU, memory and session statistics). A statistics stage computes intervals, paired comparisons, bottleneck labels and the queueing model. A final stage generates every figure, table and number in this paper as LaTeX macros, so no result is typed by hand.

## VI. Experimental Methodology

### A. Testbed

The client runs on a c7i.4xlarge instance (Intel Xeon Platinum 8488C, 16 vCPU, 31 GiB) and MinIO, as a single node in Docker with host networking, on an i4i.4xlarge instance (16 vCPU, local NVMe), both in one cluster placement group in AWS region ap-south-2. Before the runs we measured 12.4 Gbit/s (1,479 MiB/s) between the hosts with iperf3, a 0.33 ms round trip, and 2,044 MiB/s read and 882 MiB/s write on the MinIO disk with fio. The experiments ran code at commits `4597849`, `4f6f8a1`, `ef30535`.

### B. Workload

PUT and GET are measured separately, mostly in a closed loop (each worker issues its next request when the previous one completes). Concurrency 1–500 is measured at 64 KiB and 1 MiB; 16 MiB stops at 128 workers because 500 objects of 16 MiB in flight exceed the client's memory; object size 1 KiB–500 MiB is measured at one worker, with 128 MiB at 32 and 500 MiB at 16 workers. Large objects require at least 10 measured operations per run. For RQ3, an open-loop generator offers Poisson arrivals of 1 KiB and 64 KiB PUTs, served by 256 threads, at rates around the measured capacity. N1 is measured at $B_{\max} \in \{10, 100, 1000\}$, $T_{\max} = 60\,\mathrm{s}$, and the queue-aware variant at $B_{\min} = 1$, $B_{\max} = 100$.

### C. Metrics

We report the four metrics of the study and one derived metric:

- *Handshake overhead*: the time spent in key establishment per request (encapsulation, RSA unwrapping, Diffie–Hellman or decapsulation), from the stage timers.
- *Throughput*: successful requests, or MiB, per second of the measured window.
- *CPU utilization*: CPU of the client's benchmark processes and of MinIO as a percentage of each host's 16 vCPU during the measured window, and the client thread CPU time per request.
- *Memory growth*: peak resident memory over the run and its growth from the first sample, for the client processes and for MinIO.
- *Response time* (open loop): queue wait plus service time per request.

### D. Replication and Statistics

Every configuration runs five times, and the order of modes is shuffled within each repetition. A run is the unit of analysis: we report the mean over runs with a Student-$t$ 95 % interval. Modes are compared only within one batch, paired by repetition, as the ratio $\exp(\overline{\log(x_a/x_b)})$ with a $t$-interval; a comparison is *equivalent* if its whole interval lies within $\pm 5\,\%$, *different* if it lies entirely outside, and *inconclusive* otherwise. Above one worker, stage timers include time spent waiting for the Python interpreter lock, so we report key-establishment cost and stage shares at one worker and client CPU as thread CPU time per request.

### E. Batches

The study ran in four batches (Table III). In batches A–C the pause between runs, meant to wait until MinIO stopped writing, fell back to 5 s because MinIO in Docker does not export its disk-write counter. A PUT run that followed another PUT run was then sometimes slowed by 12–24 %, consistent with the previous run's writeback and object purge still in progress: 38 small-object PUT cells exceeded our 5 % limit on run-to-run variation and 17 runs were flagged as stalls. Because mode order was shuffled, this added noise but no bias. Batch D re-measured all PUT cells up to 64 KiB with a 40 s pause, covering the kernel's 30 s dirty-page writeback and MinIO's 5 s cleanup interval: the variation of P1 at 64 KiB and 128 workers fell from 11.7 % to 0.7 % (B1: 10.9 % to 0.3 %), with 3 stalls in 260 runs. Batch D is the result of record for those cells.

TABLE III
BATCHES AND THE RESULTS EACH ONE SUPPLIES

| Batch | Runs | Requests | Result of record for |
|---|---|---|---|
| A | 850 | 35.4 M | concurrency sweep: GET; PUT ≥1 MiB; hybrid P2 |
| B | 360 | 1.0 M | object size 1 KiB–500 MiB; handshake cost |
| C | 500 | 46.9 M | open loop (RQ3); N1 on GET; queue-aware N1 |
| D | 260 | 18.0 M | PUT ≤64 KiB incl. N1 (40 s pause) |

TABLE IV
KEY ESTABLISHMENT PER REQUEST (MEAN [95 % CI], ONE WORKER, OBJECTS UP TO 16 MIB, BATCH B) AND MANIFEST SIZE

| Mode | PUT (ms) | GET (ms) | Manifest (B) |
|---|---|---|---|
| B0 plaintext | – | – | 522 |
| B1 AES only | 0.001 [0.001, 0.001] | 0.001 [0.001, 0.001] | 576 |
| R1 RSA-4096 | 0.094 [0.094, 0.094] | 1.486 [1.484, 1.488] | 1,276 |
| B2 X25519 | 0.087 [0.086, 0.088] | 0.049 [0.049, 0.049] | 622 |
| P1 ML-KEM-768 | 0.045 [0.045, 0.046] | 0.047 [0.046, 0.047] | 2,034 |
| P2 hybrid | – | – | 2,091 |

## VII. RESULTS

### A. Handshake Overhead (RQ1)

Table IV gives key-establishment time per request. ML-KEM-768 costs 0.045 ms to encapsulate on PUT and 0.047 ms to decapsulate on GET. X25519 costs 1.9× more on PUT, where the sender generates an ephemeral key, and about the same on GET. RSA-4096 is cheap to encrypt but costs 1.486 ms to decrypt, 32× ML-KEM decapsulation, which places its cost on the read path. The price of ML-KEM is metadata: its manifest is 2,034 bytes against 622 for X25519 and 1,276 for RSA-4096, because it carries the 1,088-byte ciphertext.

**Finding 1.** Per-object ML-KEM-768 costs tens of microseconds, less than X25519 on PUT and far less than RSA-4096 decryption on GET; its cost is metadata, not computation.

### B. Effect of Object Size (RQ2)

Fig. 2 shows median latency against object size at one worker. For P1 the share of request time spent in ML-KEM falls from 1.41 % (PUT) and 2.19 % (GET) at 1 KiB to 0.012 % and 0.010 % at 500 MiB; even a 1 KiB PUT takes 3.18 ms, dominated by two storage round trips (data, then the committing manifest). The only mode visibly apart is R1 on GET: its median latency is 73 % above P1 at 64 KiB and 34 % at 1 MiB.

**Finding 2.** Storage round trips dominate request time at every object size; ML-KEM's share is largest for the smallest objects and never exceeds a few percent.

### C. Concurrency and Bottlenecks (RQ2)

Fig. 3 shows throughput from 1 to 500 workers. All modes reach the same ceilings. GETs of 1 MiB and more saturate the network at 1,477 MiB/s, 100 % of the iperf3 baseline; 1 MiB PUTs level off at about 1,310 MiB/s. Small PUTs level off

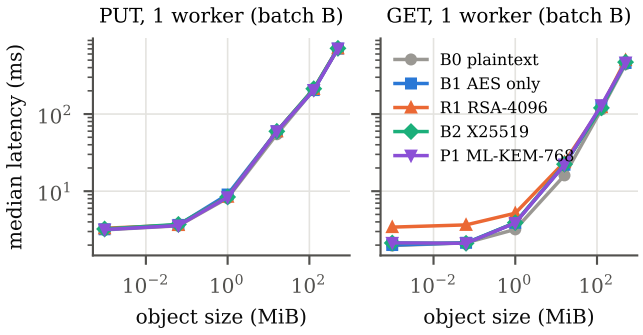



Fig. 2. Median latency against object size, one worker (batch B; log scales).

at about 4,700 requests/s (295 MiB/s at 64 KiB) while the client uses at most 49 % of its CPU, the server 72 % and the network 21 % of the link: by elimination the limit is MinIO's write path, possibly including its CPU, since 72 % of 16 vCPU leaves little headroom on 8 physical cores with hyper-threading. Small GETs are limited by the client CPU (93 % for P1 at 128 workers, 9,400 requests/s). RSA-4096 reaches that limit much earlier: at 98 % client CPU it delivers 3,600 requests/s, 0.38–0.59 of P1's throughput at 8–500 workers. For objects of 1 MiB and more, where the network or storage is the bottleneck, R1 and P1 are equivalent from 32 workers on.

**Finding 3.** Every workload is limited by the network, the storage server or the client CPU, and all modes reach the same limit—except RSA-4096, whose decryption makes small-object reads client-bound at less than two thirds of ML-KEM's throughput.

### D. ML-KEM against Classical Key Establishment (H1)

Table V summarizes the paired throughput ratios P1/B2 over the 44 closed-loop cells of record. 38 are equivalent within ±5 %, 6 inconclusive and 0 different; equivalent ratios lie between 0.961 and 1.039. At the small-PUT ceiling the ratio is 0.999 [0.993, 1.006]. In 7 of the equivalent cells the interval still excludes 1, so a small difference exists inside the margin; the largest is where the client CPU is the bottleneck (64 KiB GET, 500 workers): P1 delivers 3.9 % less than B2 [0.959, 0.964]. The inconclusive cells lie at the knee of the throughput curve (8–32 workers) or at one worker, where a single slow run widens the interval.

**Finding 4.** Replacing X25519 with ML-KEM-768 leaves throughput unchanged within ±5 % wherever the comparison is conclusive.

### E. CPU and Memory (RQ1)

Table VI lists CPU and memory for representative workloads. Per request, P1 uses between −0.045 and 0.048 ms more client CPU than X25519 at one worker, and 1.37–1.63 ms less than RSA-4096 on GET. Server CPU does not depend on the mode. Client memory is dominated by the interpreter and buffers: about 2,060 MiB at 500 workers, within 2.5 % across modes; its growth during a run tracks throughput (R1, the slowest on GET, grows least), and streaming keeps the growth of a single 500 MiB PUT at most 20 MiB. Server memory

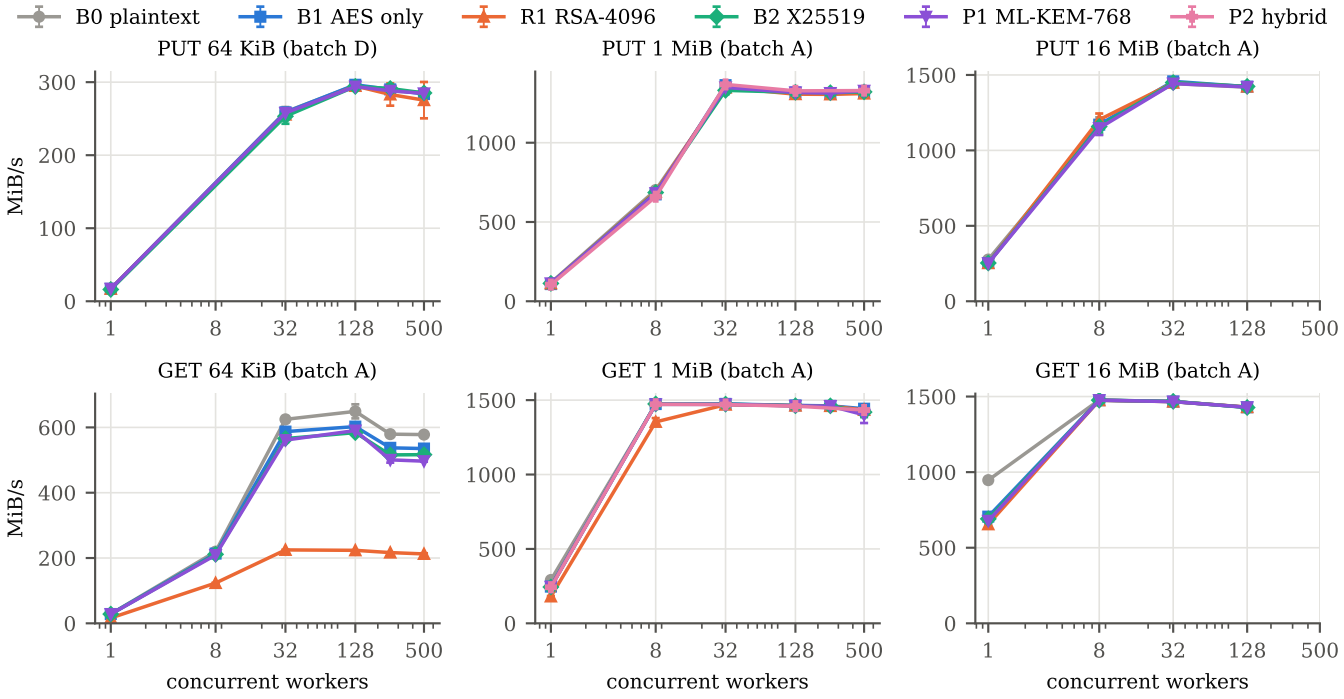



Fig. 3. Throughput (MiB/s, mean and 95 % CI over five runs) against concurrent workers. 64 KiB PUT from batch D, others from batch A. The hybrid P2 was measured at 1 MiB only.

TABLE V
P1 AGAINST X25519 (B2): PAIRED THROUGHPUT RATIOS AND VERDICTS (EQUIVALENT / INCONCLUSIVE / DIFFERENT, MARGIN ±5 %) OVER THE CELLS OF RECORD

| Batch | Op | Sizes | Workers | Eq./Inc./Diff. | P1/B2 throughput |
|---|---|---|---|---|---|
| A | GET | 1 MiB, 16 MiB | 1, 8, 32, 128, 256, 500 | 9/1/0 | 0.981–1.005 |
| A | GET | 64 KiB | 1, 8, 32, 128, 256, 500 | 6/0/0 | 0.961–1.011 |
| A | PUT | 1 MiB, 16 MiB | 1, 8, 32, 128, 256, 500 | 9/1/0 | 0.990–1.014 |
| B | GET | 1 MiB, 16 MiB, 128 MiB, 500 MiB | 1 | 3/1/0 | 0.921–1.011 |
| B | GET | 1 KiB, 64 KiB | 1 | 2/0/0 | 0.991–1.000 |
| B | PUT | 1 MiB, 16 MiB, 128 MiB, 500 MiB | 1 | 4/0/0 | 1.000–1.039 |
| D | PUT | 1 KiB, 64 KiB | 1, 32, 128, 256, 500 | 5/3/0 | 0.989–1.060 |

growth comes mostly from single runs in which MinIO grew its heap (P1 at 1 MiB PUT: one run 756 MiB, the others at most 20 MiB); its intervals overlap across modes in 4 of 4 workloads, so it does not depend on the key-establishment mode.

**Hybrid mode.** P2 (X25519+ML-KEM-768) is equivalent to P1 in 7 of the 8 cells at 1 MiB from 8 workers on. At one worker its PUT throughput is 7.3 % lower. Most of that gap, like the higher cost of B1 at the same size, sits in the AES-GCM stage (0.37 ms for P2 and 0.29 ms for B1 against 0.12 ms for P1), although all modes run the identical cipher code: it arises outside the cipher and we did not isolate its cause. At 1 MiB we therefore use B0 and B2, not B1, as references.

**Finding 5.** ML-KEM's CPU cost per request is within a few hundredths of a millisecond of X25519's, and neither client nor server memory depends on the key-establishment mode.

### F. Bounded Session Amortization

With a bound $B_{\max}$, N1 performed one ML-KEM operation per $B_{\max}$ objects (0.0100 per request at $B_{\max} = 100$), as designed and as the validator checked for every session. Fig. 4 shows the effect. At one worker N1 saves 30–66 µs of client CPU per request against P1, matching the 38–42 µs of key-establishment time it removes; with 32–128 workers the saving grows to 49–100 µs, because each ML-KEM call through the Python binding costs more CPU under contention. All these intervals exclude zero. Because ML-KEM is a small part of every request, this is visible as throughput only where the client CPU is the bottleneck: small GETs at 32–128 workers (client CPU 92 %) run 1.4–5.1 % faster. On PUT, where MinIO limits throughput, N1 is equivalent to P1 in 3 of 6 cells and inconclusive in the rest. The queue-aware variant adapts as intended: under open-loop load of 1,500 and 3,500 requests/s its mean bound was 18.4 and 31.9, so it re-keyed 5.5× and 3.4× as often as the fixed bound of 100, with a 99th-percentile response-time ratio of 1.02 [0.98, 1.07] and 0.94 [0.67, 1.33];

TABLE VI
CPU (% OF THE HOST'S 16 VCPU, MEASURED WINDOW), CLIENT CPU TIME PER REQUEST AND MEMORY (PEAK RSS AND GROWTH OVER THE RUN), MEAN OF FIVE RUNS

| Workload | Mode | Client CPU % | Server CPU % | CPU ms/req | Client peak MiB | Client growth MiB | Server growth MiB |
|---|---|---|---|---|---|---|---|
| GET 64 KiB c128 (A) | B1 | 91 | 71 | 1.49 | 2,144 | 367 | 4 |
| GET 64 KiB c128 (A) | R1 | 98 | 23 | 4.36 | 1,911 | 125 | 5 |
| GET 64 KiB c128 (A) | B2 | 92 | 69 | 1.54 | 2,134 | 349 | 0 |
| GET 64 KiB c128 (A) | P1 | 93 | 70 | 1.56 | 2,110 | 331 | 1 |
| PUT 1 MiB c128 (A) | B1 | 34 | 53 | 3.92 | 2,283 | 265 | 16 |
| PUT 1 MiB c128 (A) | R1 | 42 | 51 | 4.97 | 2,192 | 116 | 37 |
| PUT 1 MiB c128 (A) | B2 | 35 | 52 | 4.06 | 2,298 | 126 | 33 |
| PUT 1 MiB c128 (A) | P1 | 35 | 51 | 4.07 | 2,291 | 254 | 158 |
| PUT 500 MiB c16 (B) | B1 | 25 | 47 | 675.42 | 2,698 | 174 | 70 |
| PUT 500 MiB c16 (B) | R1 | 25 | 47 | 676.55 | 2,719 | 188 | 27 |
| PUT 500 MiB c16 (B) | P1 | 26 | 47 | 680.14 | 2,729 | 202 | 18 |
| GET 500 MiB c16 (B) | B1 | 23 | 6 | 1,033.11 | 2,011 | 12 | 90 |
| GET 500 MiB c16 (B) | R1 | 23 | 6 | 1,024.33 | 2,025 | 28 | 144 |
| GET 500 MiB c16 (B) | P1 | 22 | 6 | 997.48 | 2,011 | 33 | 67 |

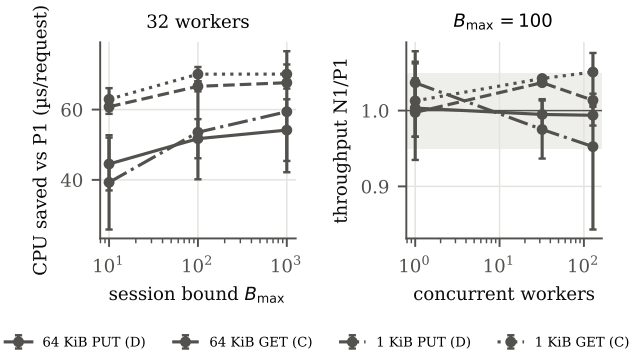



Fig. 4. N1 against P1: client CPU saved per request (left, 32 workers) and paired throughput ratio at $B_{max} = 100$ (right; shaded band ±5 %). PUT from batch D, GET from batch C.

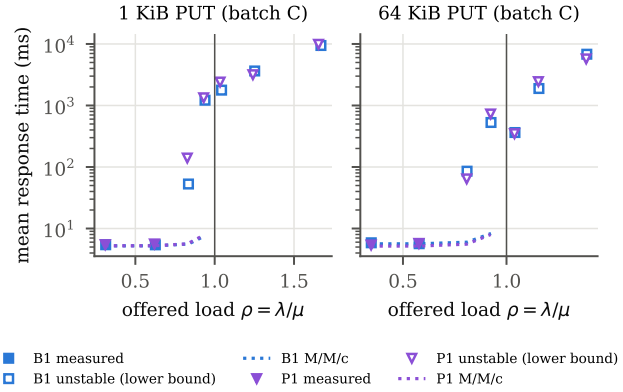



Fig. 5. Open-loop mean response time against offered load (batch C, log scale). Hollow markers: unstable runs, whose response time grows with run length (lower bounds). Dotted: M/M/c prediction.

under overload its bound rose to 100.0. This shows that the policy works at no measurable cost; it does not show that it is better than a smaller fixed bound.

**Finding 6.** Session amortization removes ML-KEM's CPU cost but improves throughput only where the client CPU is the bottleneck; elsewhere it enlarges the compromise scope for no measurable gain.

### G. Queueing Behaviour (RQ3)

Fig. 5 shows mean response time (queue wait plus service) against offered load $\rho = \lambda/\mu$, with capacity $\mu$ taken from the most overloaded runs (4,800 requests/s at 1 KiB, 4,300 at 64 KiB). We model the system as M/M/c with $c = \mu S_0$ servers, where $S_0$ is the service time at light load (5.2 ms; $c = 25$ at 1 KiB, 22 at 64 KiB). Below the knee the model is within 7–10 % of the measurements. It misses the knee: at 1 KiB and 4,000 requests/s ($\rho = 0.83$) it predicts 5.6 ms while the system was already unstable, with mean response above 140 ms; at 64 KiB the same happens at $\rho = 0.81$ (predicted 5.6 ms, measured above 64 ms). The knee lies between $\rho = 0.63$ and 0.83 at 1 KiB and between 0.58 and 0.81 at 64 KiB, for B1 and P1 alike. The model fails because service time is not independent of load: the median service time of a 1 KiB PUT rises from 5.0 ms at light load to about 50 ms in overload, as concurrent writes contend inside MinIO. Usable capacity is therefore between 0.58 and 0.83 of the saturation throughput, and ML-KEM does not move it.

**Finding 7.** Response time collapses well before nominal saturation, for classical and post-quantum modes alike; an M/M/c model with load-independent service time fits below the knee but does not predict it.

## VIII. DISCUSSION

**Implications for migration.** For per-object envelope encryption on an S3-compatible store, ML-KEM-768 is not a performance problem. Its key establishment costs tens of microseconds, less than X25519 on PUT in our stack, and disappears against the milliseconds of storage round trips; on every workload we measured, throughput is set by the network, by MinIO's write path or, for small GETs, by the client CPU, and ML-KEM reaches those limits together with the classical modes. Operators can therefore plan a migration around compatibility and key management rather than capacity.

**Where the cost lies.** The measurable costs of migration are elsewhere: 1,412 more bytes of metadata per object than X25519, which matters for very small objects and for object-count-based billing, and a few tens of microseconds of client CPU per request, which matters only when the client is saturated. RSA-4096, in contrast, is a poor baseline to stay on: its private-key operation makes small-object reads CPU-bound at 0.38–0.59 of the throughput of ML-KEM.

**When to amortize.** Session amortization is useful exactly where the client CPU is the bottleneck, such as many small reads from a few clients; elsewhere it trades compromise scope for no measurable gain, and per-object ML-KEM is the simpler and safer default.

**Capacity planning.** The open-loop results matter more than any cryptographic choice: response time collapses well before nominal saturation, so provisioning should target the knee rather than the peak throughput.

## IX. Threats to Validity

**Internal validity.** The pause between runs failed in batches A–C (§VI); batch D repeated the affected cells with a working pause, but 8 of its 52 cells still exceed 5 % variation. Absolute small-object throughput differed by up to 13 % between batches, so all comparisons are within one batch. Stage timers above one worker include interpreter-lock waiting. Five repetitions give $t$-intervals with four degrees of freedom; the attribution of the small-PUT limit to MinIO's write path is by elimination, and the AES-GCM cost difference between modes at 1 MiB is unexplained.

**External validity.** The client is Python, so absolute costs include interpreter overhead and differ from native or hardware-accelerated implementations; MinIO ran as a single node without erasure coding; results describe one instance pair in one region. liboqs is intended for prototyping, not production.

**Construct validity.** "Handshake overhead" is per-object key establishment, not a network handshake; transport security was held constant.

## X. Conclusion and Future Work

We designed, implemented and validated an instrumented, reproducible benchmark of post-quantum envelope encryption for S3-compatible object storage and measured it from 1 to 500 concurrent workers and from 1 KiB to 500 MiB objects on two separate cloud instances. ML-KEM-768 adds 0.045–0.047 ms per object, is statistically equivalent to X25519 within $\pm 5$ % in 38 of 44 comparisons, with none confirmed to differ by more, while RSA-4096 limits small-object reads to 0.38–0.59 of ML-KEM's throughput. Bounded session amortization helps only when the client CPU is the bottleneck, and a queueing model with load-independent service time does not predict where response time collapses. Future work includes native and hardware-accelerated clients, distributed MinIO with erasure coding across several servers, post-quantum signatures for object integrity, and a queueing model with load-dependent service time.

## Artifact Availability

Code, configurations, per-run summaries, the raw-data validator and the analysis scripts that generate every figure, table and number in this paper (`analysis/summarize.py`, `final_analysis.py`, `make_paper_assets.py`) are in the project repository, organized following ACM artifact guidelines [19]. The raw request records are archived separately.

## References


[1] National Institute of Standards and Technology, "Module-lattice-based key-encapsulation mechanism standard," NIST, Federal Information Processing Standards Publication FIPS 203, Aug. 2024.

[2] C. Paquin, D. Stebila, and G. Tamvada, "Benchmarking post-quantum cryptography in TLS," in *Post-Quantum Cryptography (PQCrypto 2020)*, 2020, cryptology ePrint Archive, Paper 2019/1447.

[3] D. Sikeridis, P. Kampanakis, and M. Devetsikiotis, "Post-quantum authentication in TLS 1.3: A performance study," in *Network and Distributed System Security Symposium (NDSS)*, 2020.

[4] F. Gadban and J. Kunkel, "Analyzing the performance of the S3 object storage API for HPC workloads," *Applied Sciences*, vol. 11, no. 18, p. 8540, 2021.

[5] H. Krawczyk and P. Eronen, "HMAC-based extract-and-expand key derivation function (HKDF)," RFC 5869, 2010.

[6] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC," NIST, Tech. Rep. SP 800-38D, 2007.

[7] K. Moriarty, B. Kaliski, J. Jonsson, and A. Rusch, "PKCS #1: RSA cryptography specifications version 2.2," RFC 8017, 2016.

[8] A. Langley, M. Hamburg, and S. Turner, "Elliptic curves for security," RFC 7748, 2016.

[9] MinIO, Inc., "MinIO object storage server, release RELEASE.2025-10-15T17-29-55Z (repository archived april 2026)," https://github.com/minio/minio, 2025.

[10] M. Harchol-Balter, *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*. Cambridge University Press, 2013.

[11] Open Quantum Safe Project, "liboqs: C library for prototyping and experimenting with quantum-resistant cryptography, version 0.16.0," https://github.com/open-quantum-safe/liboqs, 2026.

[12] N. Rodriguez-Alvarez, "Performance and storage analysis of CRYSTALS-Kyber as a post quantum replacement for RSA and ECC," arXiv:2508.01694, 2025.

[13] D. Connolly, P. Schwabe, and B. Westerbaan, "X-Wing: General-purpose hybrid post-quantum KEM," IETF Internet-Draft draft-connolly-cfrg-xwing-kem, 2024.

[14] L. F. Prikutse, R. E. Turkson, A.-S. Yussiff, A.-L. Yussiff, and M. G. Asante-Mensah, "A commitment-based hybrid post-quantum cryptographic model for multi-file cloud storage," arXiv:2608.13138, 2026.

[15] Amazon Web Services, "Reducing the cost of SSE-KMS with Amazon S3 Bucket Keys," Amazon S3 User Guide, https://docs.aws.amazon.com/AmazonS3/latest/userguide/bucket-key.html, 2026.

[16] ——, "Data key caching details," AWS Encryption SDK Developer Guide, https://docs.aws.amazon.com/encryption-sdk/latest/developer-guide/data-caching-details.html, 2026.

[17] Open Quantum Safe Project, "liboqs-python: Python 3 bindings for liboqs, version 0.16.0.1," https://github.com/open-quantum-safe/liboqs-python, 2026.

[18] Python Cryptographic Authority, "cryptography, version 50.0.1," https://cryptography.io, 2026.

[19] Association for Computing Machinery, "Artifact review and badging, version 1.1," https://www.acm.org/publications/policies/artifact-review-and-badging-current, 2020.